

Documentação do Código do Robô UAILL-E

Lucas Lemos Ricaldoni

Março
2025

Sumário

1	Introdução	3
2	Dependências	3
3	Estrutura do Código	4
4	Explicação das Funcionalidades	4
4.1	line_detection.py	4
4.1.1	calculate_error()	4
4.1.2	identify_colour()	5
4.1.3	verify_curve	7
4.1.4	detect_line	8
4.1.5	Exemplo de utilização	9
4.2	ball_detection.py	10
4.2.1	identify_colour	10
4.2.2	find_ball	11
4.2.3	Exemplo de utilização	13
4.3	main.py	13
4.4	robot_control.py	13
4.5	PID.c	14
4.6	motors.c	14
5	Referências Bibliográficas	15

1 Introdução

Este documento apresenta a documentação do código desenvolvido para o projeto de um **seguidor de linha** da **categoria avançada** na modalidade **Resgate da Olimpíada Brasileira de Robótica 2025**.

A lógica de detecção da linha na pista e dos desafios é baseada em conceitos de **visão computacional**. A detecção é realizada a partir de uma **câmera**, que utiliza algoritmos de visão computacional, como **detecção de bordas**, combinados com lógica de programação para permitir que o robô percorra corretamente o caminho.

Além disso, na fase de **resgate**, o sistema é projetado para **detectar bolas**, identificando suas **posições** e **cores**, garantindo que o robô consiga cumprir sua missão com precisão.

2 Dependências

O código foi desenvolvido em **Python** e em **C**. Para o desenvolvimento do sistema utilizamos as seguintes bibliotecas:

- NumPy
- OpenCV
- Matplotlib
- Ctypes
- time
- pigpio

Os modelos de visão computacional foram desenvolvidos a partir de testes realizados no ambiente de **Jupyter Notebooks**.

3 Estrutura do Código

O projeto está organizado da seguinte forma:

```
1  OBR2025/  
2  |-- c_files/  
3  |   |-- motors.c  
4  |   |-- motors.h  
5  |   |-- PID.c  
6  |   |-- PID.h  
7  |-- datas/  
8  |   |-- lines/  
9  |   |-- balls/  
10 |-- notebooks/  
11 |   |-- line-detection.ipnyb  
12 |   |-- model-to-detect-balls.ipnyb  
13 |-- pdfs/  
14 |   |-- line-detection.pdf  
15 |   |-- model-to-detect-balls.pdf  
16 |   |-- documentation.pdf  
17 |-- main.py  
18 |-- robot_control.py  
19 |-- line_detection.py  
20 |-- ball_detection.py
```

4 Explicação das Funcionalidades

Cada função possui sua docstring que facilita o entendimento do que ela realiza. Abaixo terá as explicações das principais funções e suas utilizações. Além da explicação de que cada arquivo impacta no robô.

4.1 line_detection.py

Este arquivo é responsável pela detecção das linhas, a partir de algoritmos de visão computacional que possibilita a detecção das linhas. Abaixo terá as funções presentes no arquivo.

4.1.1 calculate_error()

```

1 def calculate_error(target_line):
2     if isinstance(target_line, tuple):
3         target_line = target_line[0]
4
5     # Convert to numpy array e remove the extra dimensions
6     contour = np.squeeze(np.array(target_line))
7
8     # avoid erro if has few points
9     if contour.shape[0] < 2:
10        return None
11
12    # get the extrames points
13    x1, y1 = tuple(contour[np.argmin(contour[:, 1])]) # few
14    value of Y (top)
15    x2, y2 = tuple(contour[np.argmax(contour[:, 1])]) #
16    bigger value of Y (base)
17
18    # calculate angle em rad
19    theta_rad = np.arctan2(y2 - y1, x2 - x1)
20
21    # convert to degree
22    theta_deg = np.degrees(theta_rad)
23
24    # assure the degrees in [0, 180]
25    theta_deg = theta_deg + 180 if theta_deg < 0 else
26    theta_deg
27
28    # return the erro in degree (for less use of memory)
29    return int(theta_deg - 90)

```

Função 1: Cálculo do erro.

Esse trecho de código é utilizado para calcular o erro do robô em relação a linha. O erro é calculado a partir da inclinação do robô em relação a linha de referência (target_line). Tal erro é retornado em graus.

4.1.2 identify_colour()

```

1 def identify_colour(img):
2     # convert image in HSV scale
3     hsv_img = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
4
5     # definy colour ranges
6     color_ranges = {
7         "red": [(np.array([0, 100, 100]), np.array([10, 255,
8         255])),
9         (np.array([160, 100, 100]), np.array([180,
10        255, 255]))], # Red (Hue 0-10)

```

```

9      "green": [(np.array([40, 40, 40]), np.array([90, 255,
10      255]))], # Green (Hue 40-90)
11      "gray": [(np.array([0, 0, 50]), np.array([130, 60,
12      220]))] # Gray
13  }
14
15  kernel = np.ones((3, 3), np.uint8)
16  # create a dict to storage the colours
17  detected_areas = {}
18
19  # find colours
20  for color, ranges in color_ranges.items():
21      # create mask colours
22      mask = sum(cv2.inRange(hsv_img, lower, upper) for
23      lower, upper in ranges)
24      # reduce the noisy
25      mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
26
27      contours, _ = cv2.findContours(mask, cv2.
28      RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
29      detected_areas[color] = sum(cv2.contourArea(c) for c
30      in contours)
31
32  if contours:
33      # drawn the contours in black colour
34      cv2.drawContours(img, contours, -1, (0,0,0), 2)
35
36  if max(detected_areas.values()) > 0:
37      dominant_color = max(detected_areas, key=
38      detected_areas.get)
39
40      if dominant_color == 'green':
41          # split the image in two sides
42          mid = img.shape[1] // 2
43
44          # create a green mask
45          green_mask = sum(cv2.inRange(hsv_img, lower,
46          upper) for lower, upper in color_ranges['green'])
47
48          # calculate moments of image
49          moments = cv2.moments(green_mask)
50
51          # find the center of object
52          cX = int(moments["m10"] / moments["m00"])
53
54          # find the side
55          side = 'left' if cX < mid else 'right'
56
57          # return the green colour and your side

```

```

51         return dominant_color, side
52
53         # return the colour with most area on the image
54         return dominant_color, None
55     # return none if not has colour in the image
56     return None

```

Função 2: Identificação de cores.

Este trecho de código é responsável pela detecção das cores presentes na pista, excluindo o preto e branco, focando em cores como vermelho, verde e cinza, que determinam o trajeto a ser seguido. Com base em uma escala de cores previamente definida, é possível utilizar "máscaras" para separar as áreas da imagem que correspondem a essas cores específicas.

Além de realizar a segmentação de cores, o código inclui funções de visualização que desenham as áreas identificadas com as cores selecionadas, facilitando a análise do processo. O retorno pode ser `None` caso nenhuma cor relevante seja detectada, ou a cor identificada, juntamente com sua posição na pista, caso uma correspondência seja encontrada.

Para o funcionamento da mesma, é preciso enviar a imagem como parâmetro.

4.1.3 verify_curve

```

1 def verify_curve(contours, img_widht):
2     # Split the image in 2
3     mid = img_width // 2
4
5     for contour in contours:
6         X = contour[:, 0, 0]
7
8         points_left, points_right = np.sum(X < mid), np.sum(X
9         >= mid)
10
11         # if contour > 8 probably it's not a curve 90
12         if len(contour) < 8:
13             if points_left > points_right:
14                 return 'Left'
15             if points_left < points_right:
16                 return 'Right'
17
18     # return no curve
19     return False

```

Função 3: Verifica curva de 90°

A função acima tem o objetivo de detectar se há uma interseção de 90° na pista que não está identificada por um quadrado verde. A interseção pode ser para a direita ou esquerda.

A lógica da detecção é baseada na divisão da imagem ao longo do eixo x, separando-a em dois lados (esquerdo e direito). A partir disso, calcula-se qual lado possui a maior quantidade de pontos, retornando o lado que apresenta uma distribuição maior de pontos do que o outro, indicando uma curva de 90°. Caso não haja curva, o retorno é falso.

4.1.4 detect_line

```
1 def detect_line(img):
2     # verify if img exists
3     if img is not None:
4         # Convert the image to grayscale
5         img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
6
7         # Aplly a threshold to detect only darken colours
8         _, binary = cv2.threshold(img_gray, 50, 255, cv2.
THRESH_BINARY_INV)
9
10        # reduce the noise
11        kernel = np.ones((3, 3), np.uint8)
12        binary = cv2.morphologyEx(binary, cv2.MORPH_OPEN,
kernel)
13        binary = cv2.morphologyEx(binary, cv2.MORPH_CLOSE,
kernel)
14
15        # Detect points that form a line
16        contours, _ = cv2.findContours(binary, cv2.
RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
17
18        # verify if has a symbol on the img
19        cor_detected = identify_colour(img)
20
21        # verify if has a line on the image
22        if len(contours) > 0:
23            # drawn the line target
24            cv2.drawContours(img, contours, -1, (0, 0, 255),
2)
25
26            #draw the car flow line
27            draw_line(img)
28
```

```

29         # send img_widht as img.shape[1]
30         is_curve = verify_curve(contours, img.shape[1])
31
32         # calculate the error
33         erro = calculate_error(contours)
34
35         # to avoid false-positive
36         if (erro > 0 and erro <= 10) or (erro >= -10 and
erro < 0): is_curve = False
37
38         # to avoid false-positive
39         if cor_detected or is_curve != False: return 0,
is_curve, cor_detected
40
41         # return the erro and is_curve
42         return erro, is_curve, cor_detected
43
44     return None, None, None

```

Função 4: Detecta a linha

A função utiliza algoritmos de visão computacional para detecção de bordas (*edge detection*) e identificação de formas (*Hough Transform*). O processo inclui a conversão da imagem para a escala de cinza, seguida da aplicação de filtros para redução de ruídos. O código executa uma lógica que detecta o número de contornos na imagem, permitindo analisar a posição da linha presente.

Essa função é a principal responsável pela lógica de **detecção de linha**. Ela identifica possíveis cores na imagem, determina as posições e formatos das linhas, detecta a presença de curvas de 90° e calcula o erro do robô em relação ao trajeto.

A função recebe apenas a imagem como parâmetro e retorna no formato de uma tupla: (erro, possibilidade_de_curva, cor_detectada). Caso uma das variáveis não seja identificado, a função retornará apenas ele como **None**.

4.1.5 Exemplo de utilização

```

1 if __name__ == '__main__':
2     path = 'datas/lines/'
3     images = [f for f in os.listdir(path) if f.endswith('.jpg
')]
4

```

```

5     # for a specified file
6     #images = ["ladrinho_embranco.jpg"]
7
8     for image_name in images:
9         print(path + image_name)
10        img = cv2.imread(path + image_name)
11        #img = cv2.imread(image_name)
12
13        erro, is_curve, has_colour = detect_line(img)
14        print(erro, is_curve, has_colour)
15
16        image_name, type_file = image_name.split('.jpg')
17        download_image(erro, path, image_name, is_curve)
18        print(f"Memory used: {psutil.Process(os.getpid()).
memory_info().rss / (1024 ** 2)} MB")
19        print('-'*35)

```

Função 5: Exemplo de utilização

Se você ainda não entendeu como a lógica funciona, teste o código acima para compreendê-la melhor. O código extrai todas as imagens de teste contendo linhas e as envia para a função *detect_line()* para realizar a análise completa da imagem.

Esse trecho imprime no terminal o **erro**, a **existência de uma curva** e se há uma **cor diferente** na imagem. Além disso, o código permite visualizar o uso de memória gasto durante a execução de cada imagem.

4.2 ball_detection.py

Este arquivo foi desenvolvido para a detecção das cores e posições das bolas na arena de resgate, usando algoritmos de visão computacional. Abaixo terá as funções presentes no arquivo.

4.2.1 identify_colour

```

1     def identify_colour(hsv_colour):
2         h, s, v = hsv_colour
3
4         if v < 50:
5             return "Black"
6         elif s < 50 and 50 <= v < 200:
7             return "Grey"
8         elif s < 50 and v >= 200:
9             return "White"

```

```

10         elif (0 <= h <= 10) or (170 <= h <= 180):
11             return "Red"
12         elif 25 <= h <= 35:
13             return "Yellow"
14         elif 35 <= h <= 85:
15             return "Green"
16         elif 100 <= h <= 130:
17             return "Blue"
18         else:
19             return "Unknown"

```

Função 6: Identifica cores

Esse trecho a partir de uma análise de uma imagem na escala **HSV** *Hue* (*matriz*), *Saturation* (*saturação*) e *Value* (*valor*) retorna a cor dominante a partir dos valores da matriz.

Caso ainda tenha uma duvida, consulte as referências bibliográficas deste pdf.

4.2.2 find_ball

```

1 def find_ball(img):
2     # resize the image into dimensions ESP32-cam
3     desired_width = 320
4     desired_height = 240
5     dim = (desired_width, desired_height)
6     img = cv2.resize(img, dim, interpolation=cv2.INTER_AREA)
7
8     # to debug
9     #print(img.shape)
10
11     # transform the img in gray-scale
12     img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
13
14     # applying filtes in the image
15     img_gray = cv2.medianBlur(img_gray, 5)
16
17     # use the function to found circles on the image
18     circles = cv2.HoughCircles(img_gray, cv2.HOUGH_GRADIENT,
19                                dp=1.5, minDist=40,
20                                param1=60, param2=30, minRadius=5,
21                                maxRadius=60)
22
23     # verify if has circles on the image
24     if circles is not None:
25         circles = np.uint16(np.around(circles))

```

```

24         for i in circles[0,:]:
25             # cordenates of the circle
26             x, y, r = i[0], i[1], i[2]
27
28             # draw the outer circle
29             cv2.circle(img,(i[0],i[1]),i[2],(0,255,0),2)
30             # draw the center of the circle
31             center_position = (i[0], i[1])
32             cv2.circle(img,(i[0],i[1]),2,(0,0,255),3)
33
34             # change the image in HSV
35             img_hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
36
37             # Extracting the mean color
38             mask = np.zeros(img_hsv.shape[:2], dtype=np.uint8
39 )
40             # create the mask
41             cv2.circle(mask, (x, y), r, 255, thickness=-1)
42
43             # calculate the medium color
44             mean_colour_hsv = cv2.mean(img_hsv, mask=mask)
45             [:3]
46
47             # get the color of the object
48             img_colour = identify_colour(mean_colour_hsv)
49
50             text_position = (x - 10, y - r - 10)
51             # writing the colour on the image
52             cv2.putText(img, img_colour, text_position,
53                 fontFace=cv2.FONT_HERSHEY_SIMPLEX,
54                 fontScale=0.6,
55                 color=(0, 255, 0), thickness=1,
56                 lineType=cv2.LINE_AA)
57             return img_colour, center_position
58         return None, None

```

Função 7: Detecta a bola

O código começa redimensionando a imagem para garantir que ela esteja na melhor escala possível. Em seguida, a imagem é convertida para a escala de cinza e um filtro é aplicado para reduzir o ruído.

A detecção dos círculos (bolas) é realizada por meio da função *HoughCircles* do OpenCV. A partir dos círculos identificados, são armazenadas as coordenadas (x, y) do centro de cada um

A detecção de cor é feita convertendo a imagem para a escala **HSV** e calculando a média dos valores nessa escala. Para fins de melhor visualização,

os círculos detectados são desenhados na imagem, e um texto indicando a respectiva cor é adicionado.

A função retorna uma tupla no formato (*cor_da_bola*, *posição_da_bola*)

4.2.3 Exemplo de utilização

```
1 if __name__ == '__main__':  
2     img = cv2.imread('datas/balls/one-yellow-ball.jpg')  
3     ball_colour, ball_position = find_ball(img)  
4     print(ball_colour, ball_position)
```

Função 8: Exemplo de utilização

Se você ainda não entendeu como a lógica funciona, teste o código acima para compreendê-la melhor. O código lê uma imagem do banco de dados e a envia para a função *find_ball()* para realizar a análise completa da imagem.

Esse trecho imprime no terminal a **cor da bola**, a **posição da bola**.

4.3 main.py

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

4.4 robot_control.py

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras

viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

4.5 PID.c

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

4.6 motors.c

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

5 Referências Bibliográficas

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

6 Conclusão

Este documento apresenta a estrutura e as funcionalidades do código, facilitando o entendimento do sistema e promovendo a democratização do mesmo. Ele proporciona liberdade e autonomia para a realização de testes e a utilização do código em diferentes contextos, permitindo uma maior flexibilidade no desenvolvimento e aprimoramento do projeto.